

Python Complete Roadmap

Basic → Intermediate → Advanced → GenAI / AI Engineering

Complete 81-chapter roadmap covering Python fundamentals, professional Python, internals, concurrency, APIs, backend development, testing, performance, and practical GenAI engineering.

How to Study

- Learn the theory, then type and run the example.
- Modify examples and intentionally create errors to understand behavior.
- Build a small project after each major section.
- Use Learn → Code → Modify → Break → Debug → Explain → Build as the learning loop.
- For a GenAI-focused path, prioritize async Python, HTTP/API engineering, Pydantic, data handling, testing, concurrency and production practices.

LEVEL 1 — PYTHON FUNDAMENTALS

1. Python Introduction & Environment

Theory / Topics: What is Python; interpreter; CPython vs PyPy; Python implementations; .py files; REPL; bytecode; .pyc; virtual environments; pip; venv; versions; PEPs; PEP 8

Example:

```
print("Hello Python")
name = "Manjeet"
print("Hello", name)
```

2. Variables & Data Types

Theory / Topics: Variables; objects; references; dynamic typing; strong typing; int; float; complex; bool; str; None

Example:

```
age = 30
salary = 75000.50
name = "Manjeet"
print(type(age))
```

3. Operators

Theory / Topics: Arithmetic; comparison; logical; assignment; bitwise; membership; identity; == versus is

Example:

```
a == b # same value
a is b # same object/reference
```

4. Input & Output

Theory / Topics: print(); input(); sep; end; f-strings; formatting; escape sequences

Example:

```
name = input("Enter name: ")
print(f"Hello {name}")
print("A", "B", sep="-", end="!")
```

5. Conditional Statements

Theory / Topics: if; elif; else; nested conditions; conditional expressions; truthy/falsy values

Example:

```
age = 25
if age >= 18:
    print("Adult")
else:
    print("Minor")
```

6. Loops

Theory / Topics: for; while; range(); break; continue; pass; nested loops; loop else

Example:

```
for i in range(1, 6):
    print(i)
```

7. Strings

Theory / Topics: Indexing; slicing; immutability; concatenation; repetition; searching; replace; split; join; formatting; encoding basics

Example:

```
text = "Python Programming"
print(text[0])
print(text[0:6])
print(text[::-1])
```

8. Lists

Theory / Topics: Indexing; slicing; mutation; nested lists; append; extend; insert; remove; pop; clear; index; count; sort; reverse; copying

Example:

```
numbers = [3, 1, 4, 2]
numbers.sort()
print(numbers)
```

9. Tuples

Theory / Topics: Immutable sequences; packing; unpacking; nested tuples; lists versus tuples

Example:

```
x, y = (10, 20)
print(x, y)
```

10. Sets

Theory / Topics: Uniqueness; hashing; union; intersection; difference; symmetric difference

Example:

```
a = {1, 2, 3}
b = {3, 4, 5}
print(a & b)
print(a | b)
print(a - b)
```

11. Dictionaries

Theory / Topics: Keys; values; hashing; insertion order; nested dictionaries; keys; values; items; get; update; pop; setdefault

Example:

```
user = {"name": "Manjeet", "age": 30}
print(user["name"])
```

12. Comprehensions

Theory / Topics: List; conditional list; set; dictionary; nested comprehensions

Example:

```
squares = [x*x for x in range(10)]
even = [x for x in range(20) if x % 2 == 0]
squares = {x: x*x for x in range(5)}
```

13. Functions

Theory / Topics: Definitions; parameters; arguments; return values; local/global scope; defaults; keywords

Example:

```
def add(a, b):
    return a + b
print(add(10, 20))
```

14. Function Arguments — Deep Dive

Theory / Topics: Positional; keyword; defaults; *args; **kwargs; keyword-only; positional-only

Example:

```
def add(*numbers):
    return sum(numbers)

def create_user(name, *, age):
    pass

def add2(a, b, /):
    return a + b
```

15. Scope & Namespace

Theory / Topics: LEGB: Local, Enclosing, Global, Built-in; global; nonlocal

Example:

```
x = 10
def test():
    x = 20
    print(x)
test()
print(x)
```

16. Modules & Packages

Theory / Topics: import; from import; aliases; module; package; subpackage; __init__.py; absolute and relative imports

Example:

```
import math
from math import sqrt
import math as m
```

LEVEL 2 — INTERMEDIATE PYTHON

17. Exception Handling

Theory / Topics: try; except; else; finally; raise; custom exceptions

Example:

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
finally:
    print("Done")
```

18. File Handling

Theory / Topics: open(); r/w/a/x; rb/wb; read; write; readline; readlines; with/context managers

Example:

```
with open("data.txt", "r") as file:
    content = file.read()
print(content)
```

19. JSON

Theory / Topics: JSON structure; dumps; loads; serialization; deserialization; Python object ↔ JSON

Example:

```
import json
data = {"name": "Manjeet", "age": 30}
text = json.dumps(data)
data2 = json.loads(text)
```

20. CSV

Theory / Topics: csv module; reader; writer; DictReader; DictWriter; headers

Example:

```
import csv
with open("users.csv") as file:
    for row in csv.reader(file):
        print(row)
```

21. Object-Oriented Programming

Theory / Topics: Class; object; constructor; instance/class variables; instance/class/static methods

Example:

```
class Employee:
    company = "ABC"
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
    def display(self):
        print(self.name, self.salary)
```

22. OOP — Core Principles

Theory / Topics: Encapsulation; abstraction; inheritance; polymorphism

Example:

```
class Animal:
    def speak(self): print("Animal sound")
class Dog(Animal):
    def speak(self): print("Bark")
```

23. Advanced OOP

Theory / Topics: Multiple inheritance; MRO; super(); abstract classes/methods; ABCs; composition; aggregation

Example:

```
class B(A):
    def show(self):
        print("B")
        super().show()
```

24. Magic / Dunder Methods

Theory / Topics: __init__; __str__; __repr__; __len__; __eq__; __lt__; __add__; __iter__; __next__; __enter__; __exit__

Example:

```
class User:
    def __init__(self, name): self.name = name
    def __str__(self): return self.name
```

25. Iterables & Iterators

Theory / Topics: Iterable vs iterator; iter(); next(); __iter__; __next__; StopIteration

Example:

```
numbers = [1, 2, 3]
it = iter(numbers)
print(next(it))
print(next(it))
```

26. Generators

Theory / Topics: yield; lazy evaluation; generator state; generator expressions; memory efficiency

Example:

```
def numbers():
    for i in range(5):
        yield i
for n in numbers(): print(n)
squares = (x*x for x in range(1000000))
```

27. Lambda Functions

Theory / Topics: Anonymous functions; transformations; appropriate versus inappropriate use

Example:

```
square = lambda x: x * x
print(square(5))
```

28. Functional Programming

Theory / Topics: map; filter; reduce; any; all; zip; enumerate; sorted

Example:

```
numbers = [1, 2, 3, 4]
result = list(map(lambda x: x*2, numbers))
```

29. Decorators

Theory / Topics: Functions as objects; higher-order functions; wrappers; @decorator; functools.wraps

Example:

```
def decorator(func):
    def wrapper():
        print("Before"); func(); print("After")
    return wrapper
@decorator
def hello(): print("Hello")
```

30. Closures

Theory / Topics: Nested functions; free variables; enclosing scope; closure state

Example:

```
def outer(x):
    def inner(y): return x + y
    return inner
add10 = outer(10)
print(add10(5))
```

31. Context Managers

Theory / Topics: with; __enter__; __exit__; contextlib; resource management

Example:

```
class MyContext:
    def __enter__(self): print("Start")
    def __exit__(self, exc_type, exc, tb): print("End")
```

32. Regular Expressions

Theory / Topics: re; patterns; character classes; quantifiers; groups; anchors; lookahead; capture groups

Example:

```
import re
text = "My phone is 9876543210"
m = re.search(r"\d{10}", text)
print(m.group())
```

33. Type Hints

Theory / Topics: Annotations; list[int]; dict[str,int]; tuple; Optional; Union; Literal; Any; TypeAlias; static checking

Example:

```
def add(a: int, b: int) -> int:
    return a + b
```

34. Advanced Type System

Theory / Topics: Protocol; TypedDict; Generic; TypeVar; Callable; ParamSpec; variance; mypy; pyright

Example:

```
from typing import Protocol
class Runnable(Protocol):
    def run(self) -> None: ...
```

35. Dataclasses

Theory / Topics: @dataclass; generated constructor; equality; repr; frozen; defaults; field()

Example:

```
from dataclasses import dataclass
@dataclass
class User:
    name: str
    age: int
```

36. Enums

Theory / Topics: Enum; members; values; API states; workflow states

Example:

```
from enum import Enum
class Status(Enum):
    ACTIVE = "active"
    INACTIVE = "inactive"
```

LEVEL 3 — ADVANCED PYTHON

37. Python Object Model

Theory / Topics: Everything is an object; identity; type; value; references; mutability; id(); type()

Example:

```
a = [1, 2]
b = a
b.append(3)
print(a)
```

38. Memory Management

Theory / Topics: Stack/heap concepts; object memory; reference counting; garbage collection; cycles; gc; weak references; lifetime

Example:

```
import sys
x = []
print(sys.getrefcount(x))
```

39. Mutable vs Immutable Objects

Theory / Topics: Immutable: int, float, bool, str, tuple, frozenset; mutable: list, dict, set, bytearray

Example:

```
x = [1, 2]
y = x
y.append(3)
print(x)
```

40. Shallow Copy vs Deep Copy

Theory / Topics: copy.copy; copy.deepcopy; nested references; when each is appropriate

Example:

```
import copy
a = [[1, 2], [3, 4]]
b = copy.copy(a)
c = copy.deepcopy(a)
```

41. Python Import System

Theory / Topics: Import resolution; sys.path; sys.modules; caching; circular imports; initialization; import hooks

Example:

```
import sys
print(sys.path)
```

42. Virtual Environments & Dependency Management

Theory / Topics: venv; pip; requirements.txt; versions; semantic versioning; conflicts; pip freeze; Poetry; uv; pip-tools

Example:

```
python -m venv .venv
pip install requests
pip freeze
```

43. Python Packaging

Theory / Topics: pyproject.toml; metadata; build systems; wheels; source distributions; src layout; publishing

Example:

```
my_package/
  pyproject.toml
  src/my_package/__init__.py
  tests/
```

44. Logging

Theory / Topics: logging; DEBUG; INFO; WARNING; ERROR; CRITICAL; handlers; formatters; file and structured logging

Example:

```
import logging
logging.basicConfig(level=logging.INFO)
logging.info("Application started")
```

45. Testing

Theory / Topics: pytest; unit/integration tests; assertions; fixtures; parameterization; mocking; isolation; coverage

Example:

```
def add(a, b): return a + b
def test_add(): assert add(2, 3) == 5
```

46. Mocking

Theory / Topics: unittest.mock; Mock; MagicMock; patch; side_effect; return_value

Example:

```
from unittest.mock import Mock
client = Mock()
client.get.return_value = {"ok": True}
```

47. Concurrency Fundamentals

Theory / Topics: Sequential vs concurrent vs parallel; I/O-bound vs CPU-bound work

Example:

```
# Select async/threading/processes based on workload characteristics
```

48. Threading

Theory / Topics: Threads; shared state; locks; race conditions; deadlocks; synchronization

Example:

```
from threading import Thread
def task(): print("Running")
t = Thread(target=task)
t.start(); t.join()
```

49. Multiprocessing

Theory / Topics: Processes; isolation; IPC; process pools; CPU-bound workloads

Example:

```
from multiprocessing import Process
def task(): print("Running")
p = Process(target=task)
p.start(); p.join()
```

50. GIL — Global Interpreter Lock

Theory / Topics: What GIL is; CPython; CPU-bound vs I/O-bound behavior; threading vs multiprocessing; performance implications

Example:

```
# Understand when the GIL limits Python threads and how processes/async I/O differ.
```

51. Async Python

Theory / Topics: async; await; asyncio; coroutines; event loop

Example:

```
import asyncio
async def task():
    await asyncio.sleep(1)
asyncio.run(task())
```

52. Asyncio Deep Dive

Theory / Topics: Event loop; task; future; awaitable; scheduling; cancellation; timeout; exceptions; async context managers/iterators; gather

Example:

```
tasks = [asyncio.create_task(task1()), asyncio.create_task(task2())]
await asyncio.gather(*tasks)
```

53. HTTP & API Programming

Theory / Topics: requests; httpx; GET/POST/PUT/PATCH/DELETE; headers; query/path params; body; JSON; status codes; auth; timeouts; retries

Example:

```
import requests
r = requests.get("https://example.com/api/users")
print(r.status_code)
print(r.json())
```

54. REST API Development

Theory / Topics: FastAPI; routes; request/response models; validation; dependency injection; middleware; exceptions; auth; async endpoints; OpenAPI

Example:

```
from fastapi import FastAPI
app = FastAPI()
@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"id": user_id}
```


55. Database Programming

Theory / Topics: SQL; SELECT/INSERT/UPDATE/DELETE; JOIN; GROUP BY; INDEX; transactions; sqlite3; SQLAlchemy; ORM; migrations; pooling

Example:

```
import sqlite3
conn = sqlite3.connect("app.db")
cur = conn.cursor()
cur.execute("SELECT 1")
```

56. Serialization & Validation

Theory / Topics: JSON; pickle awareness; dataclasses; Pydantic; validation; schemas

Example:

```
from pydantic import BaseModel
class User(BaseModel):
    name: str
    age: int
```

57. Performance Optimization

Theory / Topics: Big-O; profiling; benchmarking; memory profiling; caching; lazy evaluation; generators; timeit; cProfile; tracemalloc

Example:

```
import timeit
print(timeit.timeit("sum(range(1000))", number=10000))
```

58. Caching

Theory / Topics: lru_cache; cache; cache invalidation; TTL; Redis; application caching

Example:

```
from functools import lru_cache
@lru_cache
def fib(n):
    if n < 2: return n
    return fib(n-1) + fib(n-2)
```

59. Advanced Data Structures & Algorithms

Theory / Topics: Stack; queue; deque; linked list; trees; BST; heap; trie; graphs; BFS; DFS; shortest path; sorting; searching; recursion; DP; greedy; collections; heapq; bisect; itertools

Example:

```
from collections import deque
q = deque([1, 2, 3])
q.append(4)
print(q.popleft())
```

60. collections Module

Theory / Topics: Counter; defaultdict; deque; namedtuple; ChainMap

Example:

```
from collections import Counter
print(Counter(["AI", "Python", "AI", "API"]))
```

61. itertools

Theory / Topics: count; cycle; repeat; chain; product; permutations; combinations; groupby; islice

Example:

```
from itertools import combinations
print(list(combinations([1, 2, 3], 2)))
```

62. functools

Theory / Topics: reduce; partial; lru_cache; cache; wraps; singledispatch

Example:

```
from functools import partial
multiply = lambda a, b: a*b
double = partial(multiply, 2)
print(double(5))
```

63. Descriptors

Theory / Topics: __get__; __set__; __delete__; descriptor protocol; properties and methods as descriptor use cases

Example:

```
class Descriptor:
    def __get__(self, obj, owner):
        return "value"
```

64. Properties

Theory / Topics: @property; getter; setter; controlled attribute access

Example:

```
class Employee:
    def __init__(self, salary): self._salary = salary
    @property
    def salary(self): return self._salary
```

65. Metaclasses

Theory / Topics: Classes as objects; type; __new__; __init__; metaclass concept; framework use cases; when not to use

Example:

```
class Meta(type): pass
class MyClass(metaclass=Meta): pass
```

66. Reflection & Introspection

Theory / Topics: dir; type; id; vars; getattr; setattr; hasattr; callable; inspect

Example:

```
method = getattr(obj, "run")
if callable(method): method()
```

67. Dynamic Runtime Behavior

Theory / Topics: Dynamic attributes; monkey patching; runtime inspection; callable objects; dynamic imports; metadata

Example:

```
class User: pass
u = User()
u.name = "Manjeet"
print(u.name)
```

68. Security in Python

Theory / Topics: Input validation; secrets; environment variables; unsafe deserialization; command injection; path traversal; dependency vulnerabilities; auth

Example:

```
import os
api_key = os.getenv("API_KEY")
# Avoid hardcoded production secrets.
```

69. Environment Configuration

Theory / Topics: .env; environment variables; configuration management; secrets

Example:

```
import os
api_key = os.getenv("API_KEY")
```

70. CLI Applications

Theory / Topics: argparse; click; typer; command-line arguments; subcommands

Example:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("--name")
args = parser.parse_args()
print(args.name)
```

LEVEL 4 — PYTHON FOR GENAI / AI ENGINEERING

71. Python for LLM APIs

Theory / Topics: LLM API clients; OpenAI-compatible APIs; hosted/local models; streaming; request/response handling; timeouts; retries; errors

Example:

```
# Prompt -> API request -> model -> response -> parsing -> validation
```

72. Structured LLM Outputs

Theory / Topics: JSON; Pydantic; JSON Schema; structured outputs; validation; retry; fallback

Example:

```
class Result(BaseModel):
    answer: str
    confidence: float
```

73. Async LLM/API Calls

Theory / Topics: asyncio + HTTP + LLMs; concurrent requests; rate limits; gathering results; timeout/retry

Example:

```
results = await asyncio.gather(
    call_llm(prompt1), call_llm(prompt2), call_llm(prompt3)
)
```

74. RAG with Python

Theory / Topics: Documents; chunking; embeddings; vector DB; retrieval; context construction; reranking; generation; evaluation

Example:

```
# Documents -> chunks -> embeddings -> retrieval -> context -> LLM -> answer
```

75. Evaluation Systems

Theory / Topics: Evaluation datasets; model outputs; evaluators; correctness; relevance; faithfulness; groundedness; hallucination; context precision/recall; answer relevance; latency; tokens; cost

Example:

```
# Dataset -> model -> output -> evaluator -> metrics -> report
```

76. LLM-as-a-Judge

Theory / Topics: Judge prompts; scoring; rubrics; structured judge output; bias/variance; judge agreement

Example:

```
# Input + generated answer -> judge LLM -> score + reason
```

77. Dataset Engineering

Theory / Topics: JSONL; CSV; Parquet; pandas; transformation; filtering; deduplication; splits; golden datasets; synthetic data; eval datasets

Example:

```
import pandas as pd
df = pd.read_json("dataset.jsonl", lines=True)
print(df.head())
```

78. Observability

Theory / Topics: Prompts; responses; latency; tokens; cost; retrieval traces; metadata; errors; evaluation scores; logging; OpenTelemetry concepts; LangSmith/Langfuse concepts

Example:

```
# Capture traces needed to debug quality, cost, latency and failures.
```

79. Guardrails

Theory / Topics: Input/output validation; PII detection; prompt-injection detection; schema validation; content filtering; tool validation

Example:

```
# Validate model inputs and outputs before sensitive actions.
```

80. AI Agents

Theory / Topics: Agents; tool/function calling; state; memory; workflows; retries; human-in-the-loop; agent evaluation

Example:

```
# Agent -> select tool -> execute -> observe -> continue/finish
```

81. Production AI Application Architecture

Theory / Topics: FastAPI; AI service; retrieval; vector DB; model provider; evaluation; observability; logging; monitoring; caching; authentication; testing; deployment architecture

Example:

```
# Client -> FastAPI -> AI service -> retrieval/model -> validation -> evaluation/observability
```

Priority Guide for a GenAI-Focused Python Path

- Basic ■■■ — Syntax, control flow, strings, collections, functions.
- Intermediate ■■■■■ — Functions, OOP, iterators, generators, decorators, exceptions, typing.
- Advanced ■■■■■■ — Asyncio, concurrency, Python internals, packaging, testing, logging.
- Backend ■■■■■ — HTTP, REST, FastAPI, Pydantic, databases.
- GenAI ■■■■■■■ — LLM APIs, RAG, evaluation, LLM-as-a-Judge, datasets, agents.
- Production ■■■■■■ — Testing, observability, security, performance, deployment architecture.

Must-Master Topics

- Data types; lists/dicts/sets/tuples; functions; *args/**kwargs; scope/LEGB.
- Comprehensions; exceptions; modules/packages; OOP; dunder methods.
- Iterators; generators; decorators; closures; context managers.
- Type hints; dataclasses; Pydantic; async/await; asyncio; threading; multiprocessing; GIL.
- HTTP/REST; FastAPI; databases; testing with pytest; mocking; logging; packaging; security.
- Performance; caching; collections; itertools; functools; advanced object model.
- LLM APIs; structured outputs; RAG; evaluation; LLM-as-a-Judge; golden datasets; observability; guardrails; agents.

For an experienced Java/JavaScript/TypeScript/API/automation developer, the objective is not to relearn programming from scratch. Reach practical Python fluency quickly, then go deep on async programming, API engineering, testing, data processing and GenAI systems.